

Tree Boundary Traversal

```
class Solution {
    ArrayList<Integer> ans = new ArrayList<>();

    ArrayList<Integer> boundaryTraversal(Node root) {
        if (root == null) return ans;

        if (!isLeaf(root))
            ans.add(root.data);

        leftBoundary(root.left);
        leaves(root);
        rightBoundary(root.right);

        return ans;
    }

    boolean isLeaf(Node node) {
        return node.left == null && node.right == null;
    }

    void leftBoundary(Node node) {
        while (node != null) {
            if (!isLeaf(node))
                ans.add(node.data);

            if (node.left != null)
                node = node.left;
            else
                node = node.right;
        }
    }

    void leaves(Node node) {
        if (node == null) return;
    }
}
```

```

        if (isLeaf(node)) {
            ans.add(node.data);
            return;
        }

        leaves(node.left);
        leaves(node.right);
    }

    void rightBoundary(Node node) {
        ArrayList<Integer> temp = new ArrayList<>();

        while (node != null) {
            if (!isLeaf(node))
                temp.add(node.data);

            if (node.right != null)
                node = node.right;
            else
                node = node.left;
        }

        for (int i = temp.size() - 1; i >= 0; i--)
            ans.add(temp.get(i));
    }
}

```

Print the left view of a binary tree.

```

class solution {
    ArrayList<Integer> leftview(Node root) {
        ArrayList<Integer> ans = new ArrayList<>();
        if (root == null) return ans;

        Queue<Node> q = new LinkedList<>();
        q.add(root);
    }
}

```

```

while (!q.isEmpty()) {
    int size = q.size();

    for (int i = 0; i < size; i++) {
        Node curr = q.poll();

        if (i == 0)
            ans.add(curr.data);

        if (curr.left != null)
            q.add(curr.left);

        if (curr.right != null)
            q.add(curr.right);
    }
}

return ans;
}
}

```

Print the right view of a binary tree.

```

class solution {
    ArrayList<Integer> rightview(Node root) {
        ArrayList<Integer> ans = new ArrayList<>();
        if (root == null) return ans;

        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            int size = q.size();

            for (int i = 0; i < size; i++) {
                Node curr = q.poll();
            }
        }
    }
}

```

```

        if (i == size - 1)
            ans.add(curr.data);

        if (curr.left != null)
            q.add(curr.left);

        if (curr.right != null)
            q.add(curr.right);
    }
}

return ans;
}
}

```

Find the lowest common ancestor (LCA) of two nodes in a binary tree.

```

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null||root==p||root==q) return root;
        TreeNode left=lowestCommonAncestor(root.left,p,q);
        TreeNode right=lowestCommonAncestor(root.right,p,q);
        if(left!=null&&right!=null) return root;
        if(left==null) return right;
        else return left;
    }
}

```

Find the path from the root to a given node in a binary tree.

```

class Solution {
    public ArrayList<ArrayList<Integer>> paths(Node root) {
        ArrayList<ArrayList<Integer>> ans = new ArrayList<>();
        ArrayList<Integer> path = new ArrayList<>();
        solve(root, path, ans);
        return ans;
    }
}

```

```
void solve(Node root, ArrayList<Integer> path,
ArrayList<ArrayList<Integer>> ans) {
    if (root == null) return;

    path.add(root.data);

    if (root.left == null && root.right == null) {
        ans.add(new ArrayList<>(path));
    } else {
        solve(root.left, path, ans);
        solve(root.right, path, ans);
    }

    path.remove(path.size() - 1);
}
}
```